Audio Summarizer

Yashu Shukla
Hanyang University
Seoul, South Korea
yshukla@hanyang.ac.kr

Abstract

This project addresses the challenge of efficiently processing and summarizing audio-based news content. By converting approximately 200 Reuters News and DW news mp4 files into text using Automatic Speech Recognition system, SpeechRecognition library, we aim to improve user experience by extracting concise information from long transcripts using a strong bidirectional context understanding and autoregressive text generator, BART. Our approach seeks to facilitate quicker analysis of news topics, ultimately providing users with streamlined content.

1 Introduction.

In recent years, the rapid advancement of Natural Language Processing (NLP) has opened up new possibilities for automated text processing tasks. One such application is automatic summarization, which aims to condense long pieces of text into concise summaries. This research explores the application of NLP techniques to summarize audio content, particularly news broadcasts.

By leveraging the power of NLP, this research aims to automate the process of summarizing news audio content, making it more accessible and efficient for consumers. The generated summaries can be used to provide concise news updates, create highlights, or assist in information retrieval.

2 Literature review

Approaches used under audio summarization are as follows: directing the summary using only audio features, converting audio signals to text and directing the summarization process using textual methods and an approach which consists of a mixture of the first two [1]. Advancements in Automatic Speech Recognition (ASR) systems, such as Google Speech-to-Text, IBM Watson, and OpenAI's Whisper, have enabled accurate conversion of audio into text, particularly for structured news content. In text summarization, models like T5 (Text-to-Text Transfer Transformer) and BART (Bidirectional and Auto-Regressive Transformers) have been widely used for summarizing lengthy texts, offering high performance in NLP tasks. [2]

3 Methodology

3.1 Data collection and Preprocessing

A dataset of 200 news audio files was collected from Reuters News and DW News' Youtube channel on which speech-to-text conversion was performed using SpeechRecognition library to obtain textual transcripts. These obtained textual files were then punctuated in order to define sentence boundaries by using PunctuationModel from deepmultilingualpunctuation library.

The transcribed text was cleaned by taking reference to the various methods mentioned in class to ensure consistent formatting. Firstly, sentence tokenization was done by using the sent_tokenize

function which was imported from the nltk.tokenize library. This function splitted the input files' content into separate sentences. Sentence tokenization helps break down the text into manageable parts by identifying boundaries between sentences, which provides a clearer structure for further processing. After this, word tokenization was done by using word_tokenize function from the nltk.tokenize library. Word tokenization follows to break sentences down into individual words or subword units for deeper analysis.

```
I'm on YouTube, McKinnon, welcome to the program.
Israel says its troops have entered Southern Lebanon as its anticipated ground defensive gets underway.
now Israel's military is carrying out what it's calling targeted ground raids against Hezbollah.
```

Fig. 1: Sentence Tokenisation

```
['I', "'m", 'on', 'YouTube', ',', 'McKinnon', ',', 'welcome', 'to', 'the', 'program', '.', 'Israel', 'says', 'its', 'troops', 'have',
'entered', 'Southern', 'Lebanon', 'as', 'its', 'anticipated', 'ground', 'defensive', 'gets', 'underway', '.', 'now', 'Israel', "'s",
'military', 'is', 'carrying', 'out', 'what', 'it', "'s", 'calling', 'targeted', 'ground', 'raids', 'against', 'Hezbollah', '.', 'the',
```

Fig. 2: Word Tokenisation

Furthermore, stop words as well as noise was removed from the tokenized text by using stopwords function from nltk.corpus library. Removing them helped in reducing the noise in the data, making it easier for models to focus on the more informative words.

```
["'m", 'youtube', ',', 'mckinnon', ',', 'welcome', 'program', '.', 'israel', 'says', 'troops', 'entered', 'southern', 'lebanon',
'anticipated', 'ground', 'defensive', 'gets', 'underway', '.', 'israel', "'s", 'military', 'carrying', "'s", 'calling', 'targeted',
'ground', 'raids', 'hezbollah', '.', 'iran-backed', 'group', 'says', 'launched', 'missiles', 'towards', 'headquarters', 'israel', "'s",
```

Fig. 3: Stop Words Removed

The saved files were then lemmatized by using WordNetLemmatizer function from nltk.stem library, instead of stemming as it was, providing us with a better performance. Hence, this reduced the variability of the words by converting different inflected forms of a word to a single base form, making it easier to analyze the meaning without treating each variation as a separate entity.

```
["'m", 'youtube', ',', 'mckinnon', ',', 'welcome', 'program', '.', 'israel', 'say', 'troop', 'entered', 'southern', 'lebanon',
'anticipated', 'ground', 'defensive', 'get', 'underway', '.', 'israel', "'s", 'military', 'carrying', "'s", 'calling', 'targeted',
'ground', 'raid', 'hezbollah', '.', 'iran-backed', 'group', 'say', 'launched', 'missile', 'towards', 'headquarters', 'israel', "'s",
```

Fig. 4: Lemmatization

POS Tagging was also done to assign a grammatical category (such as noun, verb, adjective, etc.) to each word in a sentence which helps in understanding the syntactic structure of a sentence.

```
youtube: NN
,: ,
mckinnon: NN
,: ,
welcome: JJ
program: NN
.: .
israel: NNS
say: VBP
troop: NN
entered: VBD
southern: JJ
lebanon: NN
anticipated: VBN
ground: NN
defensive: JJ
get: NN
underway: RB
.: .
israel: NN
's: POS
military: JJ
carrying: NN
's: POS
calling: VBG
targeted: VBN
ground: NN
raid: NN
hezbollah: NN
.: .
```

Fig. 5: POS Tagging

3.2 Data Visualization

For better understanding, interpreting, and communicating the insights gained from textual data, we decided to visualize the contents of the raw punctual files. To begin with, we first noted the average number of words in the whole dataset came out to be 1424.88 while the average number of characters came out to be 8159.15. This analysis helped us better understand the structure of text and in the later stages, it helped us determine the ideal length for generating summaries.

```
paths = [path1, path2, path3, path4, path5, path6]
total_number_of_words = 0
total_number_of_characters = 0
count_of_file = 0
for path in paths:
    if os.path.exists(path):
        for filename in os.listdir(path):
            if filename.endswith('.txt'):
                file_path = os.path.join(path, filename)
                if os.path.isfile(file_path):
                    with open(file_path, 'r', encoding='utf-8') as file:
                        content = file.read()
                        total_number_of_words += len(content.split())
                        total_number_of_characters += len(content)
                        count_of_file += 1
    else:
        print(f"Directory not found: {path}")
if count_of_file > 0:
    average_words = total_number_of_words / count_of_file
    average_characters = total_number_of_characters / count_of_file
    print(f'Processed {count_of_file} files:')
    print(f' - Average number of words: {average_words:.2f}')
    print(f' - Average number of characters: {average_characters:.2f}')
else:
    print('No .txt files found in the directories.')
```

```
Processed 200 files:
 - Average number of words: 1424.88
 - Average number of characters: 8159.15
```

Fig.6: Code for average words and average characters in 200 files

We also analyzed the text files to gain insights, for that we utilized Counter from the collections module for the task of word frequency counting, enabling us to observe the 20 most frequent words from each file along with their counts, hence providing valuable terms in the dataset.

```
import os
from collections import Counter
input_stopwords_removed_file = '/content/drive/MyDrive/stopwords_removed_no_punc/'
for index, filename in enumerate(os.listdir(input_stopwords_removed_file), start=1):
    if filename.endswith('.txt'):
        with open(os.path.join(input_stopwords_removed_file, filename), 'r') as file:
            content = file.read().split()
            word_count = Counter(content)
            print(f'File {index}: Top 20 high frequency words:')
            top_20_words = word_count.most_common(20)
            for word, count in top_20_words:
                print(f'{repr(word)}: {count}', end=', ')
            print()
```

```
File 1: Top 20 high frequency words:
"'know',": 28, "'president',": 19, "'attempt',": 16, "'assassination',": 15, "'former',": 14, "'trump',": 11, "'campaign',": 10, "'incident',": 8, "'donald',": 7, "'another',": 7, "'golf',": 7, "'person',": 6, "'f
File 2: Top 20 high frequency words:
"'people',": 17, "'days',": 13, "'much',": 13, "'water',": 13, "'us',": 13, "'rain',": 12, "'situation',": 12, "'flooding',": 11, "'also',": 11, "'still',": 10, "'lot',": 10, "'see',": 9, "'get',": 8, "'weather',"
File 3: Top 20 high frequency words:
"'black',": 9, "'harris',": 6, "'trump',": 6, "'president',": 6, "'pennsylvania',": 6, "'know',": 5, "'well',": 5, "'lot',": 5, "'like',": 5, "'drinking',": 5, "'back',": 4, "'josh',": 4, "'rates',": 4, "'think',"
File 4: Top 20 high frequency words:
"'rain',": 7, "'like',": 7, "'flooding',": 6, "'rainfall',": 6, "'water',": 5, "'heavy',": 5, "'dry',": 5, "'eastern',": 4, "'europe',": 4, "'flood',": 4, "'still',": 4, "'also',": 4, "'one',": 4, "'events',": 4,
File 5: Top 20 high frequency words:
"'hezbollah',": 6, "'israel',": 5, "'says',": 5, "'could',": 5, "'debate',": 5, "'us',": 5, "'state',": 5, "'across',": 5, "'president',": 5, "'like',": 5, "'lebanon',": 4, "'north',": 4, "'carolina',": 4, "'vance
File 6: Top 20 high frequency words:
"'israel',": 19, "'hezbollah',": 16, "'think',": 11, "'communication',": 8, "'us',": 8, "'north',": 8, "'military',": 7, "'gaza',": 7, "'war',": 7, "'explosions',": 6, "'well',": 6, "'actually',": 6, "'israeli',":
```

Fig. 7: Code for retrieving top 20 high frequency words in each file

We decided to visualize the frequently appearing words in each file using WordCloud(Fig 8) due to an immediate sense of the most prominent terms in a dataset, helping identify key words or topics.

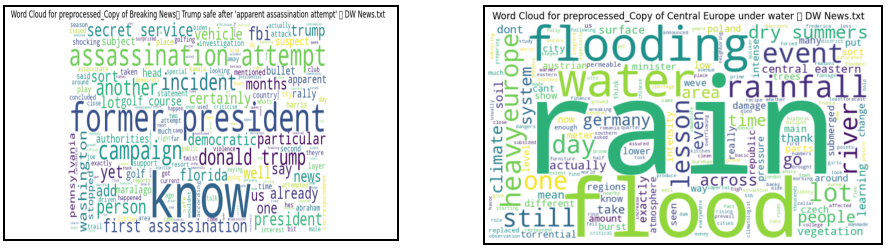


Fig. 8: WordCloud visualization

3.3 Text Representation

3.3.1 Classification and Distributed Feature Extraction Methods

In the section of classical technique for feature extraction, we used the Bag of Words (BoW) model. It converted text into a numerical representation, where each word was mapped to a unique index, which helped to transform the text into a matrix of word frequencies.

Further, we calculated TF-IDF (Term Frequency-Inverse Document Frequency) scores to identify the most significant words in each document. Words were ranked based on their importance, highlighting the top 10 words with the highest TF-IDF scores for each file (Figure 9), this helped us know the key terms for each file, along with their relevance across the dataset.

```
Top 10 TF-IDF scores for document 'Breaking News:  Trump safe after 'apparent assassination attempt' | DW News.txt':
the: 0.524878
attempt: 0.225205
that: 0.222825
to: 0.203019
assassination: 0.202381
of: 0.178260
and: 0.173309
in: 0.153502
know: 0.151587
former: 0.133534


Top 10 TF-IDF scores for document 'Central Europe under water | DW News.txt':
the: 0.446972
of: 0.231468
and: 0.207523
in: 0.191559
rainfall: 0.169323
dry: 0.168765
that: 0.159633
summers: 0.157142
rain: 0.155429
is: 0.143670
```

Fig. 9: TF-IDF scores for top 10 words of 2 files

For distributed methods, we used Fastext to generate word embeddings. As each file covered a different news topic, we selected a specific word based on the context of that news article and then identified and displayed words similar to that. This approach helped us better understand the relationships between key terms within each topic, allowing for a deeper analysis and making it easier to uncover important connections within the same news article.

3.4 Data Splitting

The input taken for the modeling were the raw punctuated text files due to our chosen model's requirements. Before moving onto that, we decided to split the dataset into train, val and test according to a ratio of 14:3:3. In short, we had 140 news text files for training, 30 files for validation and 30 files for testing.

3.5 Model Development

Our aim was to create abstractive summaries rather than extractive ones as abstractive methods generate more concise and fluent summaries by rephrasing the original content, which seems to be more suited for news text. To fulfill this task, we decided to go with the BART model (Bidirectional and Auto-Regressive Transformers), for its proven strong performance in text summarization tasks. The specific model we used is facebook/bart-large-cnn, which has 406 million parameters and is pre-trained on a large corpus of text, precisely fine-tuned for CNN/Daily Mail news articles. This makes it a powerful model, ideal for our task, which focuses on news content.

3.5.1 Creating Gold Standard Summaries for BART Training

Gold reference summaries are an important input for training the BART model. In standard practice, these summaries are typically created by humans, as human-generated summaries are often more nuanced and contextually accurate. However, given the time constraints and the immense volume of work involved in manually creating summaries for 200 news text files, it wasn't possible for us to produce handwritten (humanized) summaries. So with the advice of our mentors, we decided to leverage a large language model (LLM) to generate the summaries instead. We used the DeepAI platform for this purpose, carefully designing the prompts to specify the desired summary length and ensuring that the summaries retained a natural, humanized tone. This approach allowed us to create high-quality reference summaries efficiently.

3.5.2 Analysis of Gold Standard Summary Lengths

As our original text files had an average of 1,424 words, our goal was to produce summaries that could be read in under 2 minutes, ensuring quick updates while retaining all the important news. To accomplish this, we decided to keep the length of the gold reference summaries in the range of 200–350 words. After generating the gold standard summaries (using DeepAI), we analyzed their length and found that the average word count per file was 247.11, with an average character count of 1,721.49.

```
import os
reference_summary_folders = [
    '/content/drive/MyDrive/reference summary for train /',
    '/content/drive/MyDrive/reference summary for train /',
    '/content/drive/MyDrive/reference summary for val /',
    '/content/drive/MyDrive/reference summary for val /',
    '/content/drive/MyDrive/reference summary for test /',
    '/content/drive/MyDrive/reference summary for test /',
]
total_characters = 0
total_words = 0
file_count = 0
for folder in reference_summary_folders:
    for filename in os.listdir(folder):
        file_path = os.path.join(folder, filename)
        if os.path.isfile(file_path):
            file_count += 1
            with open(file_path, 'r', encoding='utf-8-sig') as file:
                content = file.read().strip()
                total_characters += len(content)
                total_words += len(content.split())
if file_count > 0:
    avg_characters = total_characters / file_count
    avg_words = total_words / file_count
    print(f"\nAverage number of characters per file: {avg_characters:.2f}")
    print(f"Average number of words per file: {avg_words:.2f}")
```

```
Average number of characters per file: 1721.49
Average number of words per file: 247.11
```

Fig. 10: Code for average number of characters and words per gold standard summary file

3.5.3 Training

In the training phase of the BART model, we first setup the data, which included the original punctuated news files along with their corresponding gold standard summaries. We utilized the BART tokenizer to convert the text into tokens, making sure to pad and truncate them to fit within the model's input limits—1024 tokens for the articles and 512 for the summaries. Afterwards, the Hugging Face Trainer was used to fine-tune the model. We trained the model on a GPU to speed up the training process. The hyperparameters used were , batch size, save_steps, number of epochs, and max input length. The choice of values of hyperparameters was mainly influenced by making sure that it ensures faster training while preventing issues with Google Colab's limited resources. We had to deal with the limited GPU memory while still making progress on model training. The model's progress was saved in every 500 steps and kept track of the performance. After the training was done, we saved the fine-tuned model in our drive along with the summaries the model generated.

```
import os
import torch
from transformers import BartTokenizer, BartForConditionalGeneration, Trainer, TrainingArguments
from datasets import Dataset
import gc
device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f"Using device: {device}")
train_folders = [
    '/content/drive/MyDrive/train_files_yashu/',
    '/content/drive/MyDrive/train_files_mahima/'
]
summary_folders = [
    '/content/drive/MyDrive/reference summary for train yashu/',
    '/content/drive/MyDrive/reference summary for train mahima/'
]
output_summary_folder = '/content/drive/MyDrive/please_bart_generated_summaries/'

os.makedirs(output_summary_folder, exist_ok=True)
model_name = "facebook/bart-large-cnn"
tokenizer = BartTokenizer.from_pretrained(model_name)
model = BartForConditionalGeneration.from_pretrained(model_name).to(device)

def load_and_preprocess_data(train_folders, summary_folders):
    train_texts = []
    summary_texts = []
    for train_folder, summary_folder in zip(train_folders, summary_folders):
        for filename in os.listdir(train_folder):
            if filename.endswith(".txt"):
                with open(os.path.join(train_folder, filename), 'r') as f:
                    train_texts.append(f.read())
                summary_filename = 'summary_' + filename
                summary_path = os.path.join(summary_folder, summary_filename)
                if os.path.exists(summary_path):
                    with open(summary_path, 'r') as f:
                        summary_texts.append(f.read())
                else:
                    print(f"Warning: No matching summary found for {filename}")
    return train_texts, summary_texts
train_texts, summary_texts = load_and_preprocess_data(train_folders, summary_folders)
inputs = tokenizer(train_texts, padding=True, truncation=True, max_length=1024, return_tensors="pt")
labels = tokenizer(summary_texts, padding=True, truncation=True, max_length=512, return_tensors="pt")

train_dataset = Dataset.from_dict({
    'input_ids': inputs['input_ids'],
    'attention_mask': inputs['attention_mask'],
    'labels': labels['input_ids']
})
```

```
training_args = TrainingArguments(
    output_dir='/content/drive/MyDrive/please_bart_summary_output',
    num_train_epochs=1,
    per_device_train_batch_size=2,
    save_strategy="steps",
    logging_dir='/content/drive/MyDrive/please_bart_logs_two',
    logging_steps=50,
    save_total_limit=1,
    report_to="none",
    eval_strategy="no",
    save_steps=500,
)
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset
)
trainer.train()
model.save_pretrained('/content/drive/MyDrive/please_bart_trained_model')
tokenizer.save_pretrained('/content/drive/MyDrive/please_bart_trained_model')
def generate_summaries(train_folders, summary_folders, output_summary_folder):
    for train_folder, summary_folder in zip(train_folders, summary_folders):
        for filename in os.listdir(train_folder):
            if filename.endswith(".txt"):
                # Read the text file
                with open(os.path.join(train_folder, filename), 'r') as f:
                    text = f.read()
                inputs = tokenizer(text, return_tensors="pt", truncation=True, max_length=1024).to(device)
                summary_ids = model.generate(
                    inputs['input_ids'],
                    max_length=350,
                    min_length=200,
                    num_beams=4,
                    early_stopping=True
                )
                summary = tokenizer.decode(summary_ids[0], skip_special_tokens=True)
                output_path = os.path.join(output_summary_folder, 'generated_' + filename)
                with open(output_path, 'w') as f:
                    f.write(summary)
                print(f"Generated summary saved for {filename}")
generate_summaries(train_folders, summary_folders, output_summary_folder)
gc.collect()
print("Training and summary generation completed.")
```

```
Using device: cuda
/usr/local/lib/python3.10/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
vocab.json: 100% 899k/899k [00:00<00:00, 2.74MB/s]
```

Fig. 11: Code for training the BART model

3.5.4 Validation

For validation, 30 unseen text files were used along with their gold standard summaries. In order to handle the issue of text files exceeding the model's maximum input length of 1024 tokens, we implemented a chunking mechanism, dividing the text into manageable parts. Each chunk was tokenized and then fed back into the model for summary generation. The model also used a beam search algorithm with a beam size of 2 to ensure better quality of output summaries. The generated summaries for each chunk were concatenated to produce a complete summary for the article. The validation generated summaries were saved in a specified folder in the drive.

```
import os
import torch
from transformers import BartTokenizer, BartForConditionalGeneration
import gc

device = 'cpu'
print(f'Using device: {device}')

val_folders = [
    '/content/drive/MyDrive/val_files_yashu/',
    '/content/drive/MyDrive/val_files_mahima/'
]
summary_folders = [
    '/content/drive/MyDrive/reference summary for val yashu/','/content/drive/MyDrive/reference summary for val mahima/']
output_summary_folder = '/content/drive/MyDrive/please_bart_generated_summaries_new_validation/'
os.makedirs(output_summary_folder, exist_ok=True)
model_name = "facebook/bart-large-cnn"
tokenizer = BartTokenizer.from_pretrained(model_name)
model = BartForConditionalGeneration.from_pretrained('/content/drive/MyDrive/please_bart_trained_model').to(device)
max_input_length = 1024
max_summary_length = 350
def split_text_into_chunks(text, max_length):
    tokens = tokenizer.encode(text)
    chunks = [tokens[i:i + max_length] for i in range(0, len(tokens), max_length)]
    return chunks
def generate_validation_summaries(val_folders, summary_folders, output_summary_folder):
    for val_folder, summary_folder in zip(val_folders, summary_folders):
        for filename in os.listdir(val_folder):
            if filename.endswith('.txt'):
                file_path = os.path.join(val_folder, filename)
                with open(file_path, 'r') as f:
                    text = f.read()
                if len(text.strip()) == 0:
                    print(f"Skipping empty input file: {filename}")
                    continue
                model.to('cpu')
                text_chunks = split_text_into_chunks(text, max_input_length)
                full_summary = ""
                for chunk in text_chunks:
                    chunk_text = tokenizer.decode(chunk, skip_special_tokens=True)
                    inputs = tokenizer(chunk_text, return_tensors="pt", truncation=True, max_length=max_input_length).to('cpu')
                    if inputs['input_ids'].size(1) == 0:
                        print(f"Skipping file with empty tokenization: {filename}")
                        continue
                    summary_ids = model.generate(
                        inputs['input_ids'],
                        max_length=max_summary_length,
                        min_length=200,
                        num_beams=3,
                        early_stopping=True
                    )
                    if summary_ids.size(1) == 0:
                        print(f"Summary generation failed for {filename}")
                        continue
                    summary = tokenizer.decode(summary_ids[0], skip_special_tokens=True)
                    if len(summary.strip()) == 0:
                        print(f"Generated empty summary for {filename}, skipping.")
                        continue
                    full_summary += summary + " "
                if len(full_summary.strip()) == 0:
                    print(f"Generated empty summary for {filename}, skipping.")
                    continue
                output_path = os.path.join(output_summary_folder, f'generated_val_{filename}')
                with open(output_path, 'w') as f:
                    f.write(full_summary)
                print(f"Generated summary saved for {filename}")
generate_validation_summaries(val_folders, summary_folders, output_summary_folder)
gc.collect()
print("Validation summary generation completed.")
```

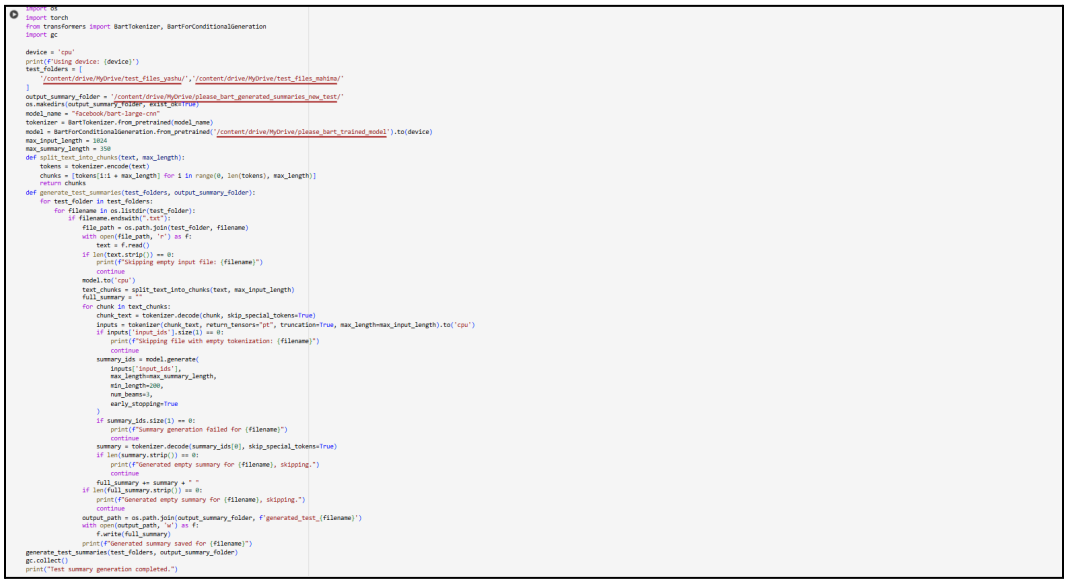
Fig. 12: Validation code

3.5.5 Model Optimization and Improvement

To improve the accuracy of the generated summaries after validation, we made some necessary adjustments to the process keeping track of the limited GPU resources. As the number of beams decides the number of different summary options the model considers before picking the optimal one, we increased the number of beams from 2 to 3 to improve the quality of generated summaries. Although we wanted to experiment with even higher beam sizes (4 or 5), again due to the limited GPU capacity of Google Collab we were not able to. Additionally, an important feature we added was the chunking process. Longer input texts were split into manageable segments of up to 1024 tokens, enabling effective processing while preserving crucial information. We also adjusted the max_length and min_length parameters for the summaries to ensure they matched the length of the gold reference summaries while maintaining coherence and relevance. All the above mentioned strategies helped improve the accuracy to a more satisfactory performance.

3.5.6 Testing

The model was finally tested on 30 unseen data to evaluate its generalization ability. The beam size, chunking strategy, and other parameters used here, were the same as those in the validation process to maintain consistency in the evaluation.



```
import os
import torch
from transformers import BartTokenizer, BartForConditionalGeneration
import gc

device = 'cpu'
print(f'Using device: {device}')
test_folders = [
    '/content/drive/MyDrive/test_files_yashu/','/content/drive/MyDrive/test_files_mahima/'
]
output_summary_folder = '/content/drive/MyDrive/please_bart_generated_summaries_new_test/'
os.makedirs(output_summary_folder, exist_ok=True)
model_name = "facebook/bart-large-cnn"
tokenizer = BartTokenizer.from_pretrained(model_name)
model = BartForConditionalGeneration.from_pretrained('/content/drive/MyDrive/please_bart_trained_model').to(device)
max_input_length = 1024
max_summary_length = 350
def split_text_into_chunks(text, max_length):
    tokens = tokenizer.encode(text)
    chunks = [tokens[i:i + max_length] for i in range(0, len(tokens), max_length)]
    return chunks
def generate_test_summaries(test_folders, output_summary_folder):
    for test_folder in test_folders:
        for filename in os.listdir(test_folder):
            if filename.endswith(".txt"):
                file_path = os.path.join(test_folder, filename)
                with open(file_path, 'r') as f:
                    text = f.read()
                if len(text.strip()) == 0:
                    print(f"Skipping empty input file: {filename}")
                    continue
                model.to('cpu')
                text_chunks = split_text_into_chunks(text, max_input_length)
                full_summary = ""
                for chunk in text_chunks:
                    chunk_text = tokenizer.decode(chunk, skip_special_tokens=True)
                    inputs = tokenizer(chunk_text, return_tensors="pt", truncation=True, max_length=max_input_length).to('cpu')
                    if inputs['input_ids'].size(1) == 0:
                        print(f"Skipping file with empty tokenization: {filename}")
                        continue
                    summary_ids = model.generate(
                        inputs['input_ids'],
                        max_length=max_summary_length,
                        min_length=200,
                        num_beams=3,
                        early_stopping=True
                    )
                    if summary_ids.size(1) == 0:
                        print(f"Summary generation failed for {filename}")
                        continue
                    summary = tokenizer.decode(summary_ids[0], skip_special_tokens=True)
                    if len(summary.strip()) == 0:
                        print(f"Generated empty summary for {filename}, skipping.")
                        continue
                    full_summary += summary + " "
                if len(full_summary.strip()) == 0:
                    print(f"Generated empty summary for {filename}, skipping.")
                    continue
                output_path = os.path.join(output_summary_folder, f'generated_test_{filename}')
                with open(output_path, 'w') as f:
                    f.write(full_summary)
                print(f"Generated summary saved for {filename}")
generate_test_summaries(test_folders, output_summary_folder)
gc.collect()
print("Test summary generation completed.")
```

Fig. 13: Testing code

4 Model Evaluation

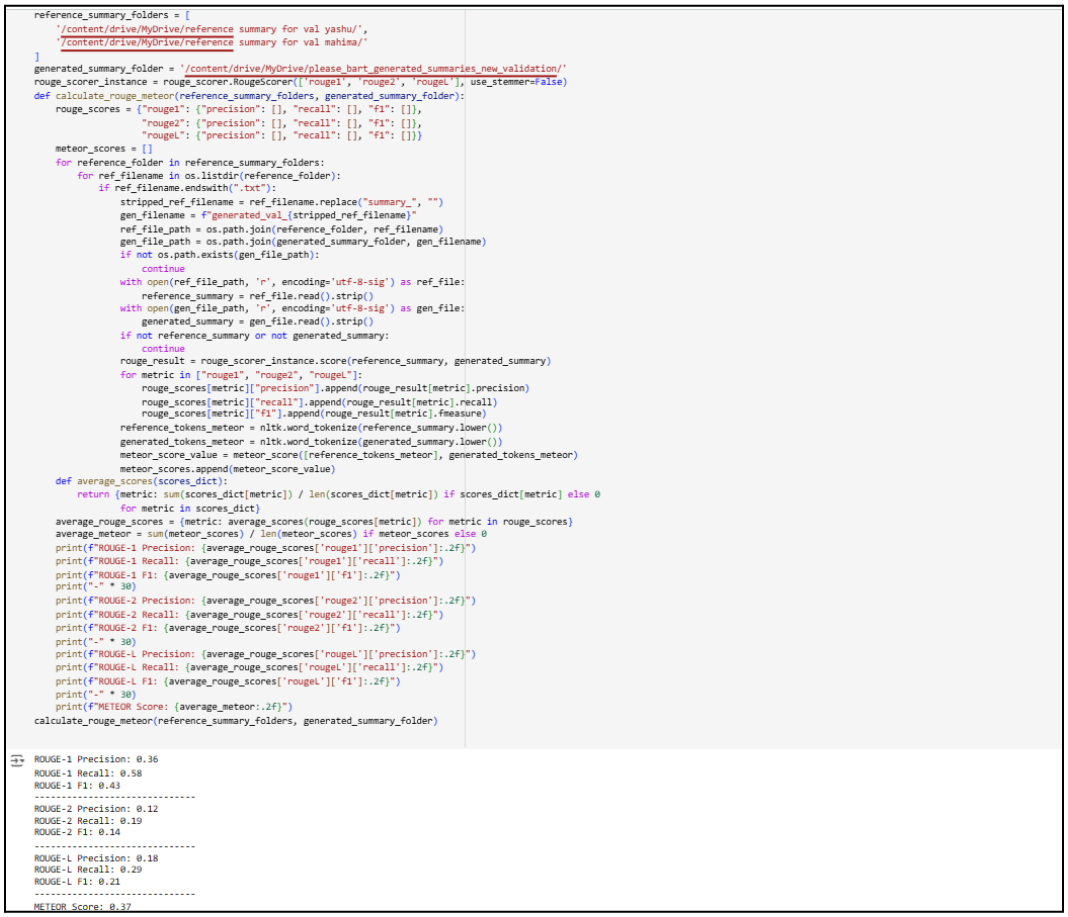
The performance of the model was evaluated using standard metrics for text summarization which are ROUGE and METEOR, as ROUGE (Recall-Oriented Understudy for Gisting Evaluation) is a set of evaluation metrics primarily used for automatic text summarization and it compares the overlap of n-grams between the generated text and reference text. ROUGE-1 (unigram based) , ROUGE-2 (bigram based) , and ROUGE-L (longest common subsequence) metrics were used which mainly focus on the precision that indicates the proportion of generated words that are relevant compared to the reference, recall which reflects the proportion of reference words captured in the generated summary and F-1 score that balances precision and recall to provide a comprehensive evaluation of the model's performance. While METEOR (Metric for Evaluation of Translation with Explicit Ordering) aims to address some of ROUGE's limitations by considering synonymy, stemming, word order, and function word matches while computing a harmonic mean of precision and recall, with additional penalties for word order mismatches, making it more sensitive to variations in phrasing and structure.

5 Results

5.1 Validation Results

The ROUGE-1 scores indicated a moderate performance in capturing unigram words, with a precision of 0.36 and recall of 0.58, and F1 score of 0.43. These results show that the model was able to retain a decent portion of the key words from the reference summaries. The ROUGE-2 scores were little on the lower side, with precision, recall, and F1 scores of 0.12, 0.19, and 0.14. It

indicates that the model struggled to capture the correct bigrams, which affects the coherence of the generated summaries. Similarly, the ROUGE-L scores showed a precision of 0.18, recall of 0.29, and F1 of 0.21, suggesting that the model lacked the ability to preserve word order and sequence. The METEOR score of 0.37, indicates a moderate level of semantic similarity between the generated and reference summaries.



```
reference_summary_folders = [
    '/content/drive/MyDrive/reference summary for val yashu/',
    '/content/drive/MyDrive/reference summary for val mahima/'
]
generated_summary_folder = '/content/drive/MyDrive/please_bart_generated_summaries_new_validation/'
rouge_scorer_instance = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'], use_stemmer=False)
def calculate_rouge_meteor(reference_summary_folders, generated_summary_folder):
    rouge_scores = {"rouge1": {"precision": [], "recall": [], "f1": []},
                    "rouge2": {"precision": [], "recall": [], "f1": []},
                    "rougeL": {"precision": [], "recall": [], "f1": []}}
    meteor_scores = []
    for reference_folder in reference_summary_folders:
        for ref_filename in os.listdir(reference_folder):
            if ref_filename.endswith(".txt"):
                stripped_ref_filename = ref_filename.replace("summary_", "")
                gen_filename = f"generated_val_{stripped_ref_filename}"
                ref_file_path = os.path.join(reference_folder, ref_filename)
                gen_file_path = os.path.join(generated_summary_folder, gen_filename)
                if not os.path.exists(gen_file_path):
                    continue
                with open(ref_file_path, 'r', encoding='utf-8-sig') as ref_file:
                    reference_summary = ref_file.read().strip()
                with open(gen_file_path, 'r', encoding='utf-8-sig') as gen_file:
                    generated_summary = gen_file.read().strip()
                if not reference_summary or not generated_summary:
                    continue
                rouge_result = rouge_scorer_instance.score(reference_summary, generated_summary)
                for metric in ["rouge1", "rouge2", "rougeL"]:
                    rouge_scores[metric]["precision"].append(rouge_result[metric].precision)
                    rouge_scores[metric]["recall"].append(rouge_result[metric].recall)
                    rouge_scores[metric]["f1"].append(rouge_result[metric].fmeasure)
                reference_tokens_meteor = nltk.word_tokenize(reference_summary.lower())
                generated_tokens_meteor = nltk.word_tokenize(generated_summary.lower())
                meteor_score_value = meteor_score([reference_tokens_meteor], generated_tokens_meteor)
                meteor_scores.append(meteor_score_value)
    def average_scores(scores_dict):
        return {metric: sum(scores_dict[metric]) / len(scores_dict[metric]) if scores_dict[metric] else 0
                for metric in scores_dict}
    average_rouge_scores = {metric: average_scores(rouge_scores[metric]) for metric in rouge_scores}
    average_meteor = sum(meteor_scores) / len(meteor_scores) if meteor_scores else 0
    print(f"ROUGE-1 Precision: {average_rouge_scores['rouge1']['precision']:.2f}")
    print(f"ROUGE-1 Recall: {average_rouge_scores['rouge1']['recall']:.2f}")
    print(f"ROUGE-1 F1: {average_rouge_scores['rouge1']['f1']:.2f}")
    print("-" * 30)
    print(f"ROUGE-2 Precision: {average_rouge_scores['rouge2']['precision']:.2f}")
    print(f"ROUGE-2 Recall: {average_rouge_scores['rouge2']['recall']:.2f}")
    print(f"ROUGE-2 F1: {average_rouge_scores['rouge2']['f1']:.2f}")
    print("-" * 30)
    print(f"ROUGE-L Precision: {average_rouge_scores['rougeL']['precision']:.2f}")
    print(f"ROUGE-L Recall: {average_rouge_scores['rougeL']['recall']:.2f}")
    print(f"ROUGE-L F1: {average_rouge_scores['rougeL']['f1']:.2f}")
    print("-" * 30)
    print(f"METEOR Score: {average_meteor:.2f}")
calculate_rouge_meteor(reference_summary_folders, generated_summary_folder)
```

```
ROUGE-1 Precision: 0.36
ROUGE-1 Recall: 0.58
ROUGE-1 F1: 0.43
------------------------------
ROUGE-2 Precision: 0.12
ROUGE-2 Recall: 0.19
ROUGE-2 F1: 0.14
------------------------------
ROUGE-L Precision: 0.18
ROUGE-L Recall: 0.29
ROUGE-L F1: 0.21
------------------------------
METEOR Score: 0.37
```

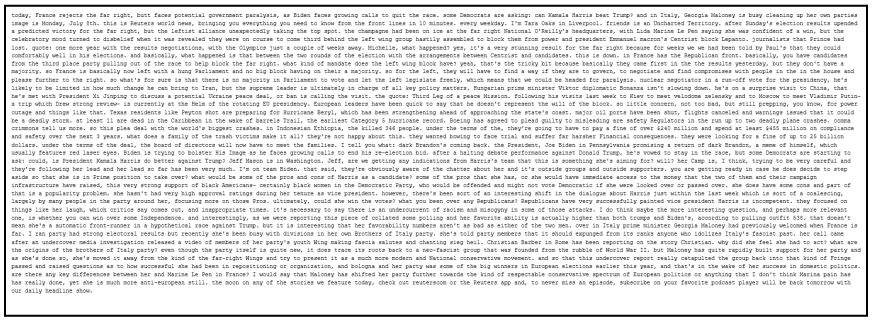
Fig. 14: Validation ROUGE and METEOR scores

5.2 Testing Results

The test results are nearly consistent with the validation results, but a slight decrease in performance is observed,as expected due to the fact that the model was evaluated on unseen data, unlike during validation where the model was evaluated with a set of reference summaries. The absence of these reference summaries in the testing phase made it more challenging for the model to produce accurate summaries, which led to slightly reduced precision, recall, and F1 scores. We can conclude that the model did show decent performance by capturing unigram overlap, but there is room for improvement in the handling of word sequences.

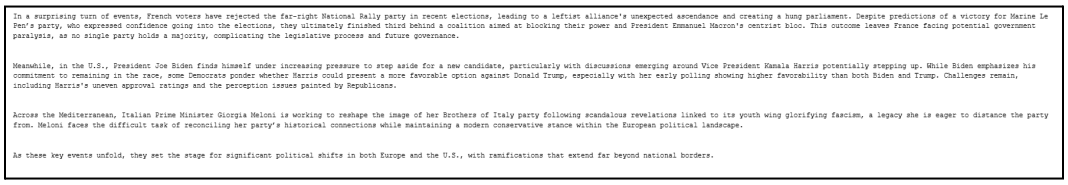
```
ROUGE-1 Precision: 0.3550
ROUGE-1 Recall: 0.5725
ROUGE-1 F1: 0.4242
--------------------------------
ROUGE-2 Precision: 0.1145
ROUGE-2 Recall: 0.1879
ROUGE-2 F1: 0.1379
--------------------------------
ROUGE-L Precision: 0.1739
ROUGE-L Recall: 0.2778
ROUGE-L F1: 0.2067
--------------------------------
METEOR Score: 0.3557
```

Fig. 15: Test ROUGE and METEOR scores



today, France rejects the far right, butt faces potential government paralysis, as Biden faces growing calls to quit the race. some Democrats are asking: can Kamala Harris beat Trump? and in Italy, Georgia Meloney is busy cleaning up her own parties image is Monday, July 8th. this is Reuters world news, bringing you everything you need to know from the front lines in 10 minutes. every weekday. I'm Tara Oaks in Liverpool. friends is an Uncharted Territory. after Sunday's election results upended a predicted victory for the far right, but the leftist alliance unexpectedly taking the top spot. the champagne had been on ice at the far right National O'Reilly's headquarters, with Lida Marine Le Pen saying she was confident of a win, but the celebratory mood turned to disbelief when it was revealed they were on course to come third behind the left wing group hastily assembled to block them from power and president Emmanuel macron's Centrist block Ispanto. Journalists that Prince had lost. quotes one more year with the results negotiations, with the Olympics just a couple of weeks away. Michelle, what happened? yes, it's a very stunning result for the far right because for weeks we we had been told by Paul's that they could comfortably well in his elections. and basically, what happened is that between the two rounds of the election with the arrangements between Centrist and candidates, this is down. in France has the Republican front. basically, you have candidates from the third place party pulling out of the race to help block the far right. what kind of mandate does the left wing block have? yeah, that's the tricky bit because basically they came first in the the results yesterday, but they don't have a majority. so France is basically now left with a hung Parliament and no big block having on their a majority. so for the left, they will have to find a way if they are to govern, to negotiate and find compromises with people in the in the house and please further to the right. so what's for sure is that there is no majority in Parliament to vote and let the left legislate freely, which means that we could be headed for paralysis. nuclear negotiator in a run-off vote for the presidency, he's likely to be limited in how much change he can bring to Iran, but the supreme leader is ultimately in charge of all key policy matters. Hungarian prime minister Viktor diplomatic Bonanza let's slowing down. he's on a surprise visit to China, that he's met with President Xi Jinping to discuss a potential Ukraine peace deal, or ban is calling the visit. the quote: Third Leg of a peace Mission. following his visits last week to Kiev to meet veldomne zelensky and to Moscow to meet Vladimir Putin- a trip which Drew strong review- is currently at the helm of the rotating EU presidency. European leaders have been quick to say that he doesn't represent the will of the block. so little concern, not too bad, but still prepping, you know, for power outage and things like that. Texas residents like Peyton shot are preparing for Hurricane Beryl, which has been strengthening ahead of approaching the state's coast. major oil ports have been shut, flights canceled and warnings issued that it could be a deadly storm. at least 11 are dead in the Caribbean in the wake of Barrels Trail, the earliest Category 5 hurricane record. Boeing has agreed to plead guilty to misleading are safety Regulators in the run up to two deadly plane crashes. comma crimmons tell us more. so this plea deal with the world's biggest crashes. in Indonesian Ethiopia, the killed 346 people. under the terms of the, they're going to have to pay a fine of over $240 million and spend at least $455 million on compliance and safety over the next 3 years. what does a family of the trash victims make it all? they're not happy about this. they wanted boeing to face trial and suffer far harsher financial consequences. they were looking for a fine of up to 25 billion dollars, under the terms of the deal, the board of directors will now have to meet the families. I tell you what: dark Brandon's coming back. the President, Joe Biden in Pennsylvania promising a return of dark Brandon, a meme of himself, which usually features red laser eyes. Biden is trying to bolster His Image as he faces growing calls to end his re-election bid. after a halting debate performance against Donald Trump, he's vowed to stay in the race, but some Democrats are starting to ask: could, is President Kamala Harris do better against Trump? Jeff Mason is in Washington. Jeff, are we getting any indications from Harris's team that this is something she's aiming for? will her Camp in, I think, trying to be very careful and they're following her lead and her lead so far has been very much. I'm on team Biden. that said, they're obviously aware of the chatter about her and it's outside groups and outside supporters. you are getting ready in case he does decide to step aside so that she is in Prime position to take over? what would be some of the pros and cons of Harris as a candidate? some of the pros that she has, or she would have immediate access to the money that the two of them and their campaign infrastructure have raised, this very strong support of black Americans- certainly black women in the Democratic Party, who would be offended and might not vote Democratic if she were locked over or passed over. she does have some cons and part of that is a popularity problem. she hasn't had very high approval ratings during her tenure as vice president. however, there's been sort of an interesting shift in the dialogue about Harris just within the last week which is sort of a coalescing, largely by many people in the party around her, focusing more on those Pros. ultimately, could she win the votes? what you been over any Republicans? Republicans have very successfully painted vice president Harris is incompetent. they focused on things like her laugh, which critics say comes out, and inappropriate times. it's necessary to say there is an undercurrent of racism and misogyny in some of those attacks. I do think maybe the more interesting question, and perhaps more relevant one, is whether you can win over some Independents. and interestingly, as we were reporting this piece of collated some polling and her favorite ability is actually higher than both trumps and Biden's, according to polling outfit 538. that doesn't mean she's a automatic front-runner in a hypothetical race against Trump. but it is interesting that her favorability numbers aren't as bad as either of the two men. over in Italy prime minister Georgia Maloney had previously welcomed when France is far. I ran party had strong electoral results but recently she's been busy with divisions in her own Brothers of Italy party. she's told party members that it should expunged from its ranks anyone who idolizes Italy's fascist past. her call come after an undercover media investigation released a video of members of her party's youth Ring making fascia salutes and chanting sieg heil. Christian Rather in Rome has been reporting on the story Christian. why did she feel she had to act? what are the origins of the Brothers of Italy party? even though the party itself is quite new, it does trace its roots back to a neo-fascist group that was founded from the rubble of World War II. but Maloney has quite rapidly built support for her party and as she's done so, she's moved it away from the kind of the far-right Wings and try to present it as a much more modern and National conservative movement, and so that this undercover report really catapulted the group back into that kind of fringe passed and raised questions as to how successful she had been in repositioning or organization, and bologna and her party was some of the big winners in European elections earlier this year, and that's in the wake of her success in domestic politics. are there any key differences between her and Marine Le Pen in France? I would say that Maloney has shifted her party further towards the kind of respectable conservative spectrum of European politics or anything that I don't think Marine pain has has really done, yet she is much more anti-european still. the moon on any of the stories we feature today, check our reutersscom or the Reuters app and, to never miss an episode, subscribe on your favorite podcast player will be back tomorrow with our daily headline show.

Fig. 16: Original punctuated text file “French election results in surprise win for the left, Biden meme and could Harris could beat Trump ? .txt”



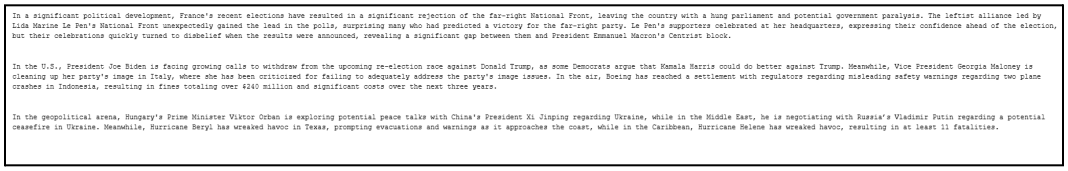
In a surprising turn of events, French voters have rejected the far-right National Rally party in recent elections, leading to a leftist alliance's unexpected ascendance and creating a hung parliament. Despite predictions of a victory for Marine Le Pen's party, who expressed confidence going into the elections, they ultimately finished third behind a coalition aimed at blocking their power and President Emmanuel Macron's centrist bloc. This outcome leaves France facing potential government paralysis, as no single party holds a majority, complicating the legislative process and future governance.

Meanwhile, in the U.S., President Joe Biden finds himself under increasing pressure to step aside for a new candidate, particularly with discussions emerging around Vice President Kamala Harris potentially stepping up. While Biden emphasizes his commitment to remaining in the race, some Democrats ponder whether Harris could present a more favorable option against Donald Trump, especially with her early polling showing higher favorability than both Biden and Trump. Challenges remain, including Harris's uneven approval ratings and the perception issues painted by Republicans.

Across the Mediterranean, Italian Prime Minister Giorgia Meloni is working to reshape the image of her Brothers of Italy party following scandalous revelations linked to its youth wing glorifying fascism, a legacy she is eager to distance the party from. Meloni faces the difficult task of reconciling her party's historical connections while maintaining a modern conservative stance within the European political landscape.

As these key events unfold, they set the stage for significant political shifts in both Europe and the U.S., with ramifications that extend far beyond national borders.

Fig. 17: Gold Standard Summary for “French election results in surprise win for the left, Biden meme and could Harris could beat Trump ? .txt”



In a significant political development, France's recent elections have resulted in a significant rejection of the far-right National Front, leaving the country with a hung parliament and potential government paralysis. The leftist alliance led by Lida Marine Le Pen's National Front unexpectedly gained the lead in the polls, surprising many who had predicted a victory for the far-right party. Le Pen's supporters celebrated at her headquarters, expressing their confidence ahead of the election, but their celebrations quickly turned to disbelief when the results were announced, revealing a significant gap between them and President Emmanuel Macron's Centrist block.

In the U.S., President Joe Biden is facing growing calls to withdraw from the upcoming re-election race against Donald Trump, as some Democrats argue that Kamala Harris could do better against Trump. Meanwhile, Vice President Georgia Maloney is cleaning up her party's image in Italy, where she has been criticized for failing to adequately address the party's image issues. In the air, Boeing has reached a settlement with regulators regarding misleading safety warnings regarding two plane crashes in Indonesia, resulting in fines totaling over $240 million and significant costs over the next three years.

In the geopolitical arena, Hungary's Prime Minister Viktor Orban is exploring potential peace talks with China's President Xi Jinping regarding Ukraine, while in the Middle East, he is negotiating with Russia's Vladimir Putin regarding a potential ceasefire in Ukraine. Meanwhile, Hurricane Beryl has wreaked havoc in Texas, prompting evacuations and warnings as it approaches the coast, while in the Caribbean, Hurricane Helene has wreaked havoc, resulting in at least 11 fatalities.

Fig.18: BART summary for “French election results in surprise win for the left, Biden meme and could Harris could beat Trump ? .txt”

6 Discussion

6.1 Model performance and Limitations

In data collection process, when we converted the audio from the different news channels to text using SpeechRecognition library, the first obstacle faced was the quality of the data being deteriorated as there were some information losses here and there. Secondly, a big concern of punctuations being missing was observed in the transcribed text. Hence, we ended up using PunctuationModel() from the deepmultilingualpunctuation library to restore them as punctuation

plays a key role in sentence segmentation and parsing. Due to time constraints and large number of files, hand written summaries could not be made for golden summary input hence we used an LLM model to generate them instead.

Another big issue faced was the availability of limited resources like gpu and drive storage. Due to BART being a model with millions of parameters, the Colab GPU was unable to run the model without crashing or hitting the limit as well as this caused the inability to use the desired parameter values during the entire process, hence we tried switching accounts and retraining the model again but it was unsuccessful because of above mentioned issue so we ended up switching to Colab Pro in order to receive some results. We took a similar road in the case of Google Drive too, by buying extra storage in order to save our files but as the trained BART model was also stored in the same Drive, we ran out of storage early on into the training process.

6.2 Future Work

To further improve the performance of the system, several potential avenues for future research can be explored like exploring advanced speech recognition techniques to improve accuracy and fluency of transcribed text. Fine-tuning the model on domain specific datasets to improve performance on specialized news domains can also be a promising approach. Developing interactive systems that allow users to provide feedback and refining the generated summaries can be adopted. Also, our data was quite limited so we expect a dataset with a larger number of files to give even more promising results, else longer audios can be extracted too instead of the average 15-20 minute ones used in the above model's chosen dataset.

Another approach to receive better results can be using hand written summaries as the gold reference summaries instead of an LLM generated summaries for training which can be a bit time consuming yet may be more accurate than the working model mentioned above.

For further advancement, the model can be extended into converting the generated concise textual summaries into concise audio summaries which can be beneficial to a large amount of population.

7 Conclusion

This research has demonstrated the feasibility of using advanced NLP techniques to automatically summarize news audio content. By leveraging the power of the BART model, we were able to generate concise and informative summaries of complex audio material.

While the results are encouraging, there is still room for improvement. Future research should focus on addressing the limitations identified in this study, such as improving the accuracy of audio-to-text transcription and enhancing the model's ability to capture nuanced information.

By continuing to push the boundaries of NLP, we can develop even more sophisticated and effective tools for information processing and dissemination.

8 References

References

[1] González-Gallardo, C. E., Deveaud, R., SanJuan, E., & Torres-Moreno, J. M. (2019, April). Audio summarization with audio features and probability distribution divergence. In *International Conference on Computational Linguistics and Intelligent Text Processing* (pp. 351-361). Cham: Springer Nature Switzerland.

[2] Abideen, Z. U. (2023, June 26). A Comparative Analysis of LLMs like BERT, BART, T5.*Medium*. https://medium.com/@zaiinn440/a-comparative-analysis-of-llms-like-bert-bart-and-t5-a4a873251ff

Gupta, N. (2020, July 13). *How Inshorts used AI to become one of India's top news apps*. WAN-IFRA. https://wan-ifra.org/2020/03/how-inshorts-used-ai-to-become-one-of-indias-top-news-apps/

P. Saxena and M. El-Haj, "Exploring Abstractive Text Summarisation for Podcasts: A Comparative Study of BART and T5 Models," in *Proc. 14th Int. Conf. Recent Advances in Natural*

Language Processing, Varna, Bulgaria, Sep. 2023, pp. 1023-1033. INCOMA Ltd., Shoumen, Bulgaria. [Online]. Available: https://aclanthology.org/2023.ranlp-1.110.

B. Yang, X. Luo, K. Sun, and M. Y. Luo, "Recent Progress on Text Summarisation Based on BERT and GPT," in *Knowledge Science, Engineering and Management*, Z. Jin, Y. Jiang, R. A. Buchmann, Y. Bi, A. M. Ghiran, and W. Ma, Eds. Cham, Switzerland: Springer, 2023, vol. 14120, Lecture Notes in Computer Science, pp. 351–366. [Online]. Available: https://doi.org/10.1007/978-3-031-40292-0_19.

H. Yadav, N. Patel, and D. Jani, "Fine-Tuning BART for Abstractive Reviews Summarization," in *Computational Intelligence*, A. Shukla, B. K. Murthy, N. Hasteer, and J. P. Van Belle, Eds. Singapore: Springer, 2023, vol. 968, Lecture Notes in Electrical Engineering, pp. 307–318.

[Online]. Available: https://doi.org/10.1007/978-981-19-7346-8_32.